



TP4 : Interprétabilité des modèles avec LIME

Objectif du TP

L'objectif de ce TP est de **comprendre et illustrer l'interprétabilité locale des modèles de deep learning** à l'aide de la méthode **LIME (Local Interpretable Model-agnostic Explanations)**, appliquée à un **problème de classification d'images**.

Nous utiliserons un modèle **InceptionV3 pré-entraîné sur ImageNet** de Keras(v2 or greater) afin d'expliquer ses prédictions sur des images.

```
import os
import keras
from keras.applications import inception_v3 as inc_net
from keras.preprocessing import image
from keras.applications.imagenet_utils import decode_predictions
from skimage.io import imread
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
print('Notebook run using keras:', keras.__version__)
```

Notebook run using keras: 2.15.0

Utilisation d'Inception :

Ici, nous créons un modèle **InceptionV3 standard pré-entraîné** et l'utilisons pour la classification d'images, après avoir effectué un **prétraitement des images à l'aide des outils de prétraitement appropriés**.

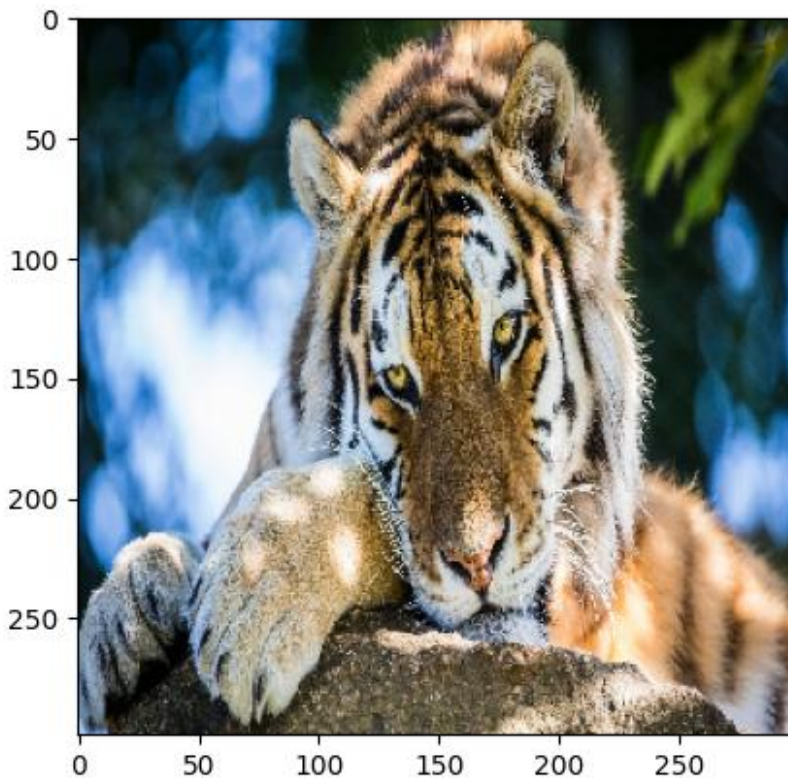
InceptionV3 est un modèle CNN profond basé sur des modules Inception, conçu pour la classification d'images et optimisé pour l'efficacité computationnelle.

```
inet_model = inc_net.InceptionV3()
```

```
def transform_img_fn(path_list):
    out = []
    for img_path in path_list:
        img = image.load_img(img_path, target_size=(299, 299))
        x = image.img_to_array(img)
        x = np.expand_dims(x, axis=0)
        x = inc_net.preprocess_input(x)
        out.append(x)
    return np.vstack(out)
```



```
images = transform_img_fn([os.path.join('data', 'Tiger-1.jpg')])
# Je divise par 2 et j'ajoute 0,5 en raison de la manière dont le modèle
Inception représente les images.
plt.imshow(images[0] / 2 + 0.5)
preds = inet_model.predict(images)
for x in decode_predictions(preds)[0]:
    print(x)
```



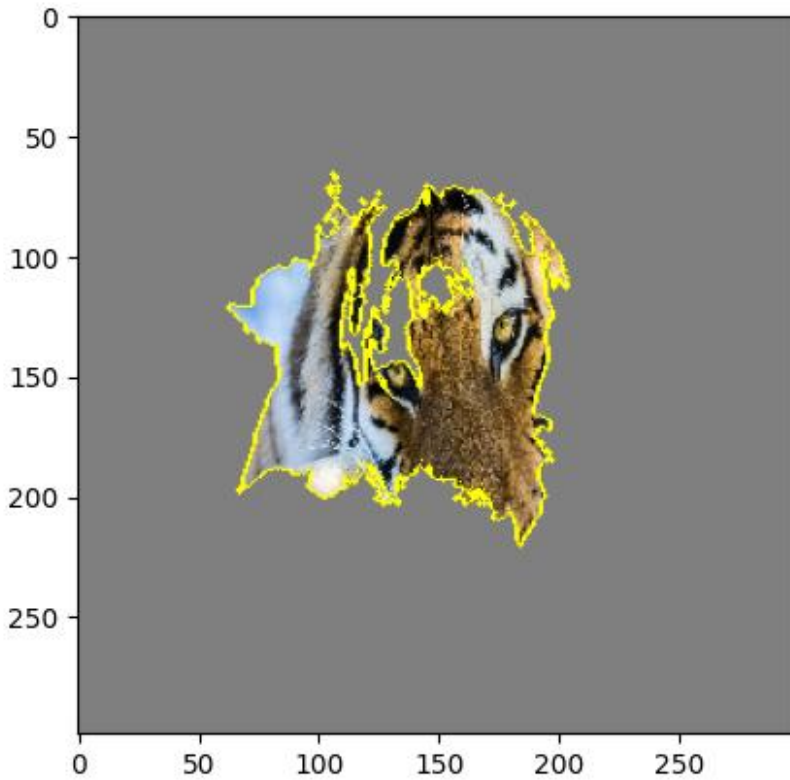
Passons maintenant à l'obtention d'une explication.

```
pip install lime
Collecting lime

%load_ext autoreload
%autoreload 2
import os,sys
try:
    import lime
```



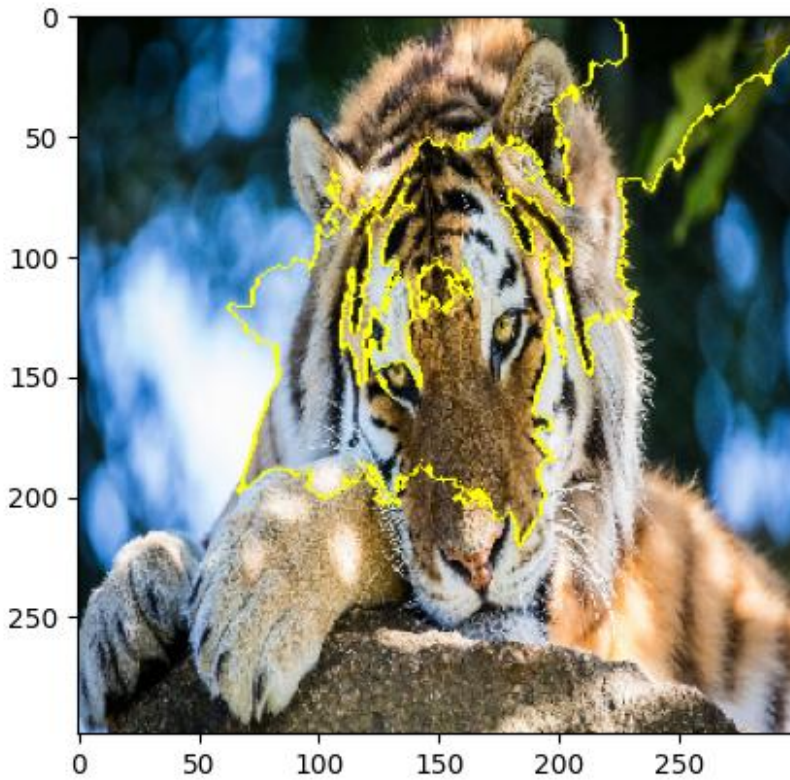
Pr S. MHAMMEDI (XAI) 2025-2026



Ou bien avec le reste de l'image visible :

```
temp, mask = explanation.get_image_and_mask(explanation.top_labels[0],  
positive_only=True, num_features=8, hide_rest=False)  
plt.imshow(mark_boundaries(temp / 2 + 0.5, mask))
```

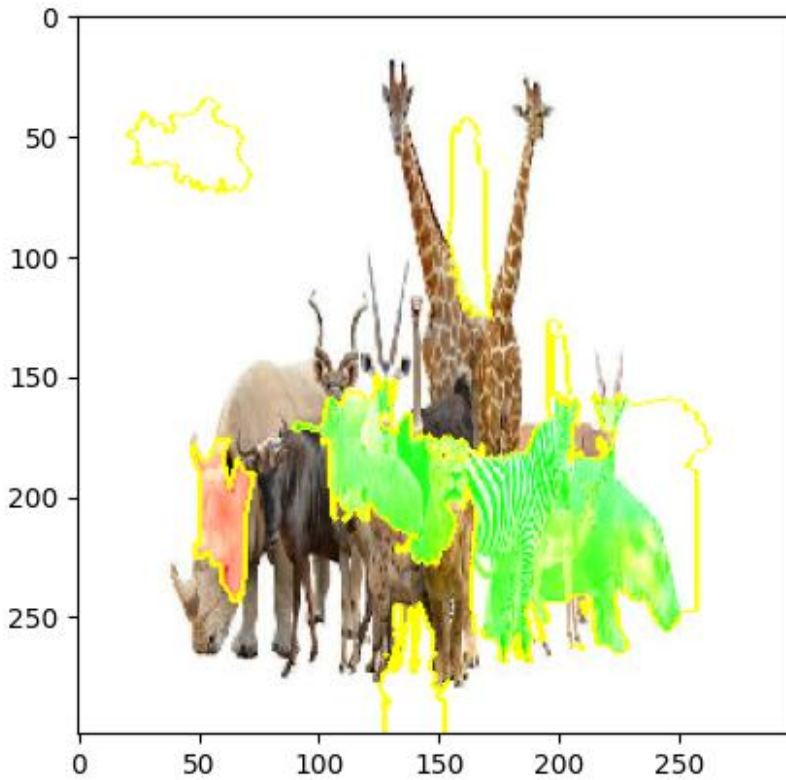
<matplotlib.image.AxesImage at 0x7d4dcce674f0>



Nous pouvons également visualiser les « avantages et inconvénients » (les avantages en vert, les inconvénients en rouge).

```
temp, mask = explanation.get_image_and_mask(explanation.top_labels[0],  
positive_only=False, num_features=10, hide_rest=False)  
plt.imshow(mark_boundaries(temp / 2 + 0.5, mask))
```

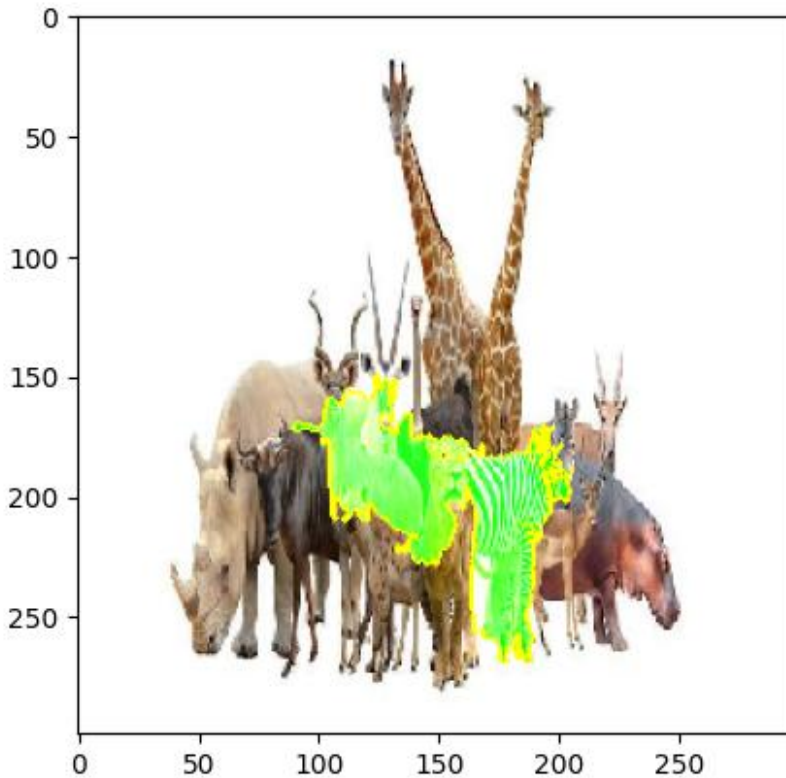
<matplotlib.image.AxesImage at 0x7d4dcd5d6aa0>



Ou bien les avantages et inconvénients dont le poids est d'au moins 0,1.

```
temp, mask = explanation.get_image_and_mask(explanation.top_labels[0],
positive_only=False, num_features=1000, hide_rest=False, min_weight=0.05)
plt.imshow(mark_boundaries(temp / 2 + 0.5, mask))
```

<matplotlib.image.AxesImage at 0x7d4dcd4585b0>



Alternativement, nous pouvons également représenter les poids des explications sous forme de carte de chaleur. La barre de couleurs indique les valeurs des poids.

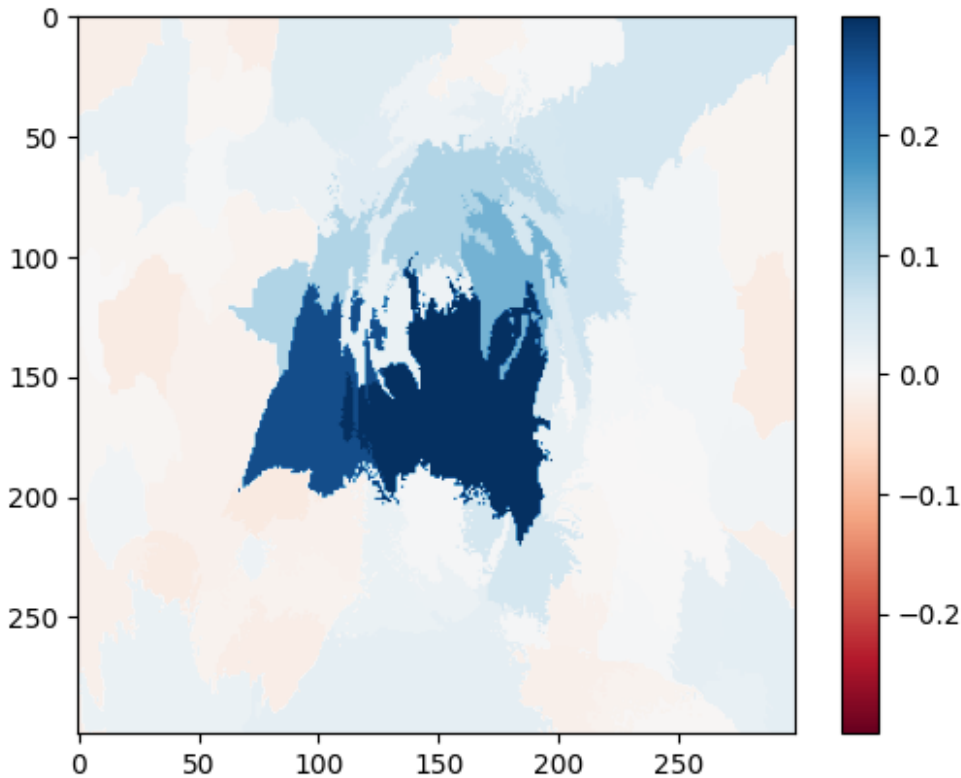
```
#Sélection de la même classe expliquée dans les figures ci-dessus.  
ind = explanation.top_labels[0]
```

```
#Associer chaque poids d'explication au super-pixel correspondant  
dict_heatmap = dict(explanation.local_exp[ind])  
heatmap = np.vectorize(dict_heatmap.get)(explanation.segments)
```

```
#Affichage. La visualisation est plus claire si une barre de couleurs  
symétrique est utilisée.
```

```
plt.imshow(heatmap, cmap = 'RdBu', vmin = -heatmap.max(), vmax =  
heatmap.max())  
plt.colorbar()
```

```
<matplotlib.colorbar.Colorbar at 0x7d4dccd0b340>
```



Voyons maintenant l'explication pour la deuxième prédiction la plus probable.

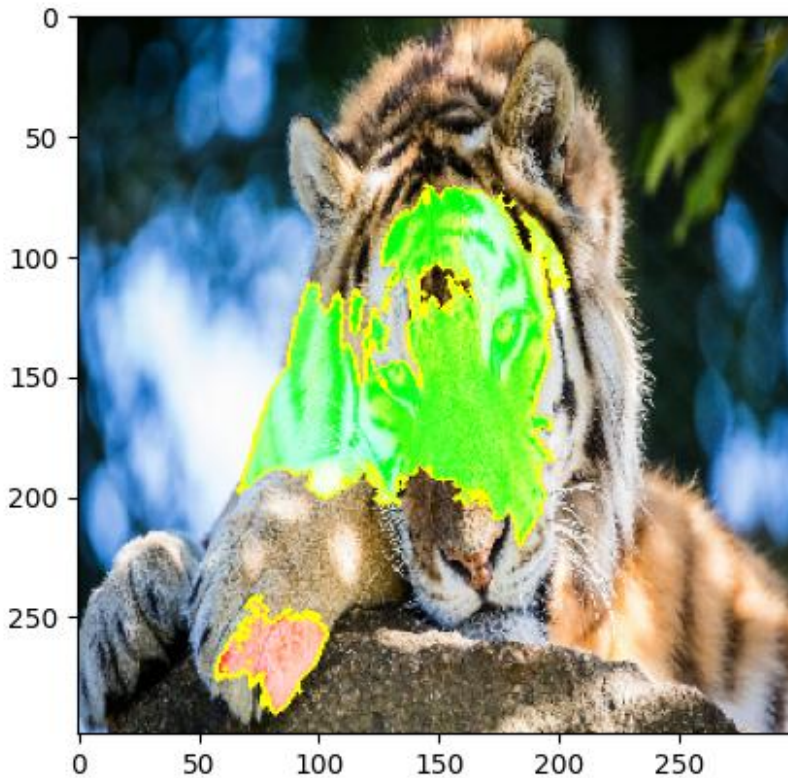
Super-pixels les plus positifs pour le wombat

```
temp, mask = explanation.get_image_and_mask(explanation.top_labels[1],  
positive_only=True, num_features=6, hide_rest=True)  
plt.imshow(mark_boundaries(temp / 2 + 0.5, mask))
```

<matplotlib.image.AxesImage at 0x7d4dccc0acb0>



```
<matplotlib.image.AxesImage at 0x7d4dccc867a0>
```



- **Pratique sur les images**

- Quels super-pixels contribuent le plus positivement à la prédiction de la classe principale ?
- Que signifie la couleur rouge et la couleur bleue dans la visualisation des super-pixels ?
- Pourquoi certains super-pixels sont masqués ou remplacés par la couleur `hide_color` ?

- **Analyse critique**

- LIME produit-il exactement la même explication à chaque exécution ? Pourquoi ?
- Quels sont les **avantages et limites** de LIME pour l'interprétation des modèles de deep learning ?
- Comment LIME pourrait-il mal représenter la contribution réelle des caractéristiques dans certains cas ?

- **Extension / réflexion avancée**

- Comment pourrait-on utiliser LIME pour expliquer plusieurs images en même temps ?
- Comment combiner LIME avec d'autres méthodes d'explicabilité (ex : SHAP) pour améliorer la confiance dans le modèle ?
- Si le modèle se trompe sur une image, que peut-on apprendre de l'explication LIME pour corriger le modèle ?